Vision and Scope

UNZA Hostel Maintenance Request and Tracking System

University of Zambia — Final Year Project

Vision and Scope

Inception phase deliverable — Unified Process (UP), per Larman's *Applying UML and Patterns*. Covers the business case, the vision, a high-level (≈10%) use-case list, and the iteration plan mapped onto UP phases. The risk list this phase also calls for is kept as its own living document — RISK_REGISTER.md — since it is *updated*, not written once, across Elaboration.

---

1. Problem statement

University of Zambia hostel residents report maintenance faults — a broken tap, a dead light, a wobbly bed — **verbally or on paper** to a caretaker or hostel office. This has four concrete, observable problems:

1. **No tracking.** Once a fault is reported there is no record of it, who is fixing it, or whether it was ever actually fixed.
2. **No accountability.** Nothing measures how long a repair took, or whether one technician is overloaded while another is idle.
3. **No visibility for the student.** A student who reported a leak has no way to check its status short of asking again in person.
4. **No data.** Estates & Maintenance cannot answer basic operational questions — which hostel has the most electrical faults, is the average repair time improving — because nothing was ever recorded in a queryable form.

2. Business case

Sponsor / primary stakeholder: UNZA Estates & Maintenance Department (represented, for this academic project, by the developer's direct contact — the administrator role in the system is that department).

Cost of the status quo: repeated verbal reports for the same fault (nobody can tell if it was already reported), faults that are forgotten entirely, no way to prioritise genuinely urgent faults (a sparking socket) over cosmetic ones (a scuffed wall), and no evidence base for planning (e.g. deciding which hostel's plumbing needs a proactive overhaul).

Benefit of the system: a single source of truth for every request, from submission to resolution, with **who is responsible, how urgent it is, and how long it has been outstanding always answerable in one query** — plus, because every transition is timestamped, the operational questions above become a report instead of a guess (see `reports.service.js` and ARCHITECTURE.md §9 for the analytics this unlocks: average resolution time, workload per technician, hostel hotspots, technician performance rankings, and recurring-fault detection).

Return on the investment is not measured in money for this academic project, but in the same terms a real IT investment would be judged on: faster resolution (an SLA clock and overdue alerts exist specifically to shorten it), fewer duplicate reports (the "Me too" upvote feature lets a second student back an existing report instead of filing a new one), and data the department did not have before (technician performance, recurring-fault detection, hostel-level trends).

3. Vision statement

> For **University of Zambia hostel residents, maintenance technicians and the Estates & Maintenance administration**, who need a reliable way to report, track and resolve hostel maintenance faults, the **UNZA Hostel Maintenance Request and Tracking System** is a **Progressive Web Application** that **replaces informal verbal/paper reporting with a tracked, role-based digital workflow** — unlike the current process, our product **gives every fault a permanent record, an owner, a target resolution time, and a status any of the three roles can check at any time**, and remains usable even on the unreliable hostel Wi-Fi students actually have.

4. Scope

In scope (this project, all delivered — see §6)

- Student self-registration and ID-based login for all roles.
- Submitting a request: category, description, hostel/room, priority, optional photo (camera capture or file upload).
- The full lifecycle `Open → Assigned → In Progress → Resolved → Closed`, enforced server-side, every transition recorded.
- Administrator triage: assignment (with a smart technician recommendation), priority changes, staff account provisioning.
- Technician workflow: see assigned work, advance status, attach proof of work.
- Student-side real-time status tracking, rating a finished repair, commenting on a request, backing ("Me too") an existing report.
- Administrator analytics: resolution time, workload, hostel hotspots, technician performance, recurring-issue detection, CSV/PDF export, a weekly emailed summary.
- Administrator communication tools: hostel-targeted announcements, preventive-maintenance scheduling.
- An in-app feedback channel (a review of the app itself) for users with no direct line to the development team.
- PWA behaviour: installable, offline request drafting with sync-on-reconnect, push notifications, dark mode.

Explicitly out of scope

- Payment or billing for repairs (all maintenance is departmentally funded; no cost tracking was requested).
- A native mobile app (a PWA was chosen specifically to avoid needing one — see CLAUDE.md's "Why These Tools").
- Multi-campus support (the schema and hostel list are UNZA Great East Road / "main" campus only, by explicit decision — see the hostel list in `client/src/utils/requestFormat.js`).
- Editing or cancelling a request after submission (an explicit, documented limitation — see ARCHITECTURE.md §14, "Future work").
- Real-time chat (comments are asynchronous, not a live chat interface).

5. Actors

Actor	Description
Student	A hostel resident. Reports faults, tracks their own requests, backs others' reports, rates finished repairs, comments, sends app feedback.
Technician	Maintenance staff. Works the jobs assigned to them, advances status, attaches proof of work, sees announcements and their own performance.
Administrator	Estates & Maintenance staff. Sees and triages every request, assigns technicians, manages staff accounts, reads analytics, posts announcements, schedules preventive maintenance, reads app feedback.
Scheduler (secondary/system actor)	Not a person — the background job that creates a request automatically when a preventive-maintenance schedule falls due (`schedules.service.runDueSchedules`). Included because it is the actor in two use cases (UC-16, UC-17 in USE_CASES.md).

6. High-level use case list (≈10%, Inception)

The handful of use cases that define *why the system exists at all* — the ones a stakeholder would name first if asked "what does this system do?". Every one of these is fully dressed in USE_CASES.md; the remaining ~90% (comments, ratings, announcements, analytics, preventive maintenance, app feedback, etc.) were elaborated across the two Elaboration iterations described in §7.

1. **Register and log in** (all actors)
2. **Submit a maintenance request** (Student)
3. **Assign a technician to a request** (Administrator)
4. **Update the status of an assigned request** (Technician)
5. **Track the status of a submitted request** (Student)

7. Iteration plan (mapped to UP phases)

Iteration	UP phase	Weeks	Goal	Evidence
—	Inception	1–2	Vision, scope, actors, the 5 core use cases above, initial risk list	This document, RISK_REGISTER.md §1
1	Elaboration	3–5	Prove the architecture end to end on the riskiest area (the layered client-server design, the ticket lifecycle) with 30% of use cases detailed	LOGICAL_ARCHITECTURE.md, DOMAIN_MODEL.md, SYSTEM_SEQUENCE_DIAGRAMS.md, commits `001` – `~030` (auth, request lifecycle, assignment)

Iteration	UP phase	Weeks	Goal	Evidence
2	Elaboration	6–8	Remaining 70% of use cases identified and detailed; refine the design class diagram; UI prototypes checked against the Stitch mockups; risk mitigation begins	USE_CASES.md, DESIGN_CLASS_DIAGRAM.md, `/design/stitch_exports`
3–4	Construction	9–12	Build every prioritised feature behind unit + integration tests, on CI, one small commit at a time	275 automated tests (TESTING.md), `.github/workflows/ci.yml`, DEFECT_LOG.md
5	Transition	13–14	Beta test with real UAT scenarios, fix what beta testing found, finish documentation	BETA_TESTING_REPORT.md, USER_MANUAL.md
—	Transition	15	Live demo, final report, repository handover	VIVA_GUIDE.md, FINAL_REPORT.md

This plan was followed **iteratively and incrementally**, not waterfall-style: features in Construction were built and demoed in small, independently-committed increments (see the actual commit history), each one adding one working, tested feature rather than one department of a big-bang design — which is the whole point of UP over a single sequential build phase.